

OSSP (Operating Systems And Systems Programming) - 25CS2104E

(SKILL Content Delivery Concepts)

1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile

CODE:

```
#include <stdio.h>
```

```
int main() {  
    printf("Linux OSSP project is ready.\n");  
    printf("GCC and Makefile can be used to build this program.\n");  
    return 0;  
}
```

```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano mkfile.c  
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc mkfile.c -o mk  
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./mk  
  
Linux OSSP project is ready.  
  
GCC and Makefile can be used to build this program.  
eeshayuktha@Eeshas-MacBook-Air OSLab %
```

To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing

CODE:

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main() {  
    pid_t pid = fork();  
    if (pid < 0) {  
        perror("fork");  
        return 1;  
    }  
    if (pid == 0) {  
        printf("Child: PID=%d PPID=%d\n", getpid(), getppid());  
        execlp("ps", "ps", "-f", (char*) NULL);  
        perror("exec");  
    } else {  
        printf("Parent: PID=%d Child PID=%d\n", getpid(), pid);  
        wait(NULL);  
        return 0;  
    }  
}
```

```
}
```

```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano proctree.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc proctree.c -o pt
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./pt
Parent: PID=78529 Child PID=78531
Child: PID=78531 PPID=78529

UID PID PPID C STIME TTY TIME CMD

502 50257 50256 0 14Aug26 ttys000 0:00.29 -zsh

502 78529 50257 0 12:29PM ttys000 0:00.01 ./pt
eeshayuktha@Eeshas-MacBook-Air OSLab %
```

2. To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

CODE:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
    char input[100];
    while (1) {
        printf("myshell> ");
        if (fgets(input, sizeof(input), stdin) == NULL)
            break;
        input[strcspn(input, "\n")] = '\0';
        if (strcmp(input, "exit") == 0)
            break;
        printf("You entered: %s\n", input);
    }
    printf("Shell exited.\n");
    return 0;
}
```

```
[eeshayuktha@Eeshas--MacBook-Air OSLab % nano bh.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc bh.c -o bh
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./bh

[Type a command: 8

Input: 8

eeshayuktha@Eeshas-MacBook-Air OSLab %
```

To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction

CODE:

```
#include <stdio.h>
#include <termios.h>
#include <unistd.h>

int main() {
    struct termios oldt, newt;
    char buffer[100];
    int len = 0;
    char ch;

    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    printf("Type a command: ");
    while (len < 99) {
        ch = getchar();
        if (ch == '\n') {
            buffer[len] = '\0';
            break;
        }
        if (ch == 127 || ch == '\b') {
            if (len > 0) {
                len--;
                printf("\b \b");
                fflush(stdout);
            }
        } else {
            buffer[len++] = ch;
            putchar(ch);
            fflush(stdout);
        }
    }

    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
    printf("\nInput: %s\n", buffer);
    return 0;
}
```

```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano bh.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc bh.c -o bh
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./bh

[Type a command: 8

Input: 8

eeshayuktha@Eeshas-MacBook-Air OSLab %
```

3. To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

CODE:

```
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 5

int main() {
    char history[MAX_HISTORY][100];
    int count = 0;
    char command[100];

    printf("Enter commands (type history to display, exit to stop).\n");
    while (1) {
        printf(">");
        if (fgets(command, sizeof(command), stdin) == NULL)
            break;
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;
        if (strcmp(command, "history") == 0) {
            for (int i = 0; i < count; i++) {
                printf("%d: %s\n", i + 1, history[i]);
            }
            continue;
        }

        if (count < MAX_HISTORY) {
            strcpy(history[count++], command);
        }
    }
    return 0;
}
```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano history.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc history.c -o history
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./history
Enter commands (type history to display, exit to stop).
>ls

>pwd

>history

1: ls

2: pwd

>exit

eeshayuktha@Eeshas-MacBook-Air OSLab %
```

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

CODE:

```

#include <stdio.h>
#include <stdlib.h>
int main() { int size = 2; int *data = malloc(size * sizeof(int)); if (data == NULL) return 1;
for (int i = 0; i < 5; i++) {
    if (i >= size) {
        size *= 2;
        data = realloc(data, size * sizeof(int));
        if (data == NULL)
            return 1;
    }
    data[i] = i + 1;
}

for (int i = 0; i < 5; i++)
    printf("%d ", data[i]);
printf("\n");

free(data);
return 0;
}
```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano realloc.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc realloc.c -o realloc
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./realloc

12345

eeshayuktha@Eeshas-MacBook-Air OSLab %
```

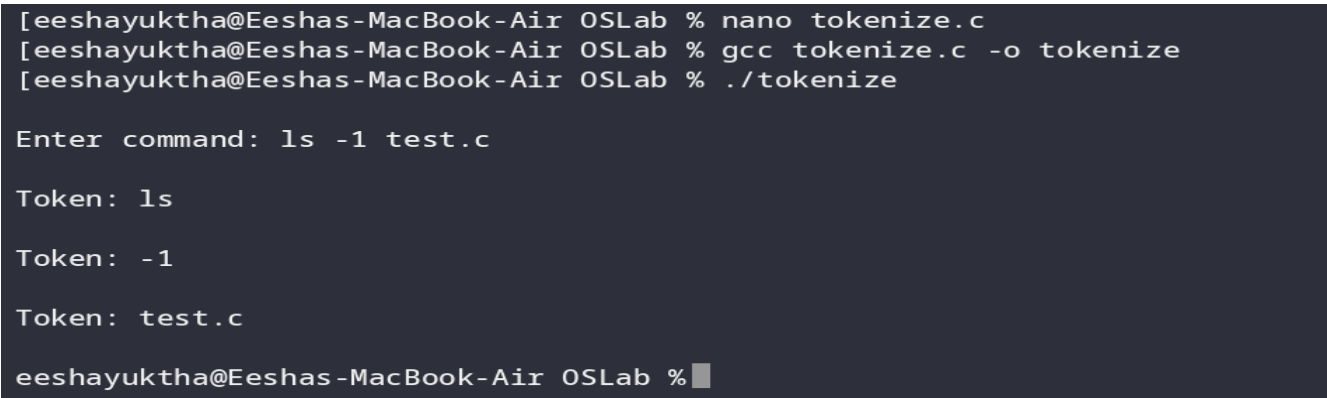
4. To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

CODE:

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[200];
    char *token;

    printf("Enter command: ");
    if (fgets(input, sizeof(input), stdin) != NULL) {
        token = strtok(input, " \\t\\n");
        while (token != NULL) {
            printf("Token: %s\\n", token);
            token = strtok(NULL, " \\t\\n");
        }
    }
    return 0;
}
```



```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano tokenize.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc tokenize.c -o tokenize
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./tokenize

Enter command: ls -l test.c

Token: ls
Token: -l
Token: test.c

eeshayuktha@Eeshas-MacBook-Air OSLab %
```

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

CODE:

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[200];
    printf("Enter command: ");
    if (fgets(input, sizeof(input), stdin) != NULL) {
        if (input[0] == '\\n') {
            printf("Empty command.\\n");
            return 0;
        }
    }
}
```

```

if (strstr(input, "&&") || strstr(input, "|"))
    printf("Logical operator detected.\n");
else if (strchr(input, '|'))
    printf("Pipeline detected.\n");
else
    printf("Simple command detected.\n");
}
return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano parser.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc parser.c -o parser
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./parser

Enter command: ls -l | grep .c

Pipeline detected.

eeshayuktha@Eeshas-MacBook-Air OSLab %

```

5. To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

CODE:

```
#include <stdio.h>
```

```

int main() {
    char input[] = "hello $USER world";
    printf("Single quoted string: %s\n", input);
    printf("Content inside single quotes is treated literally.\n");
    return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano singlequote.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc singlequote.c -o sq
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./sq

Single quoted string: 'hello $USER world'

Content inside single quotes is treated literally.
eeshayuktha@Eeshas-MacBook-Air OSLab %

```

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```

char *user = getenv("USER");
printf("Double quoted text: \"hello world\"\n");
if (user != NULL)
    printf("Expanded variable: %s\n", user);
else
    printf("USER is undefined.\n");
return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano doublequote.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc doublequote.c -o dq
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./dq

Double quoted text: "hello world"

Expanded variable: eeshayuktha

eeshayuktha@Eeshas-MacBook-Air OSLab %

```

6. To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

CODE:

```

#include <stdio.h>
#include <string.h>
int main() {
    char input[] = "hello\\ world";
    char output[100];
    int j = 0;

    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '\\' && input[i + 1] != '\0')
            i++;
        output[j++] = input[i];
    }
    output[j] = '\0';
    printf("Parsed: %s\n", output);
    return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano escape.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc escape.c -o escape
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./escape

Parsed: hello world

eeshayuktha@Eeshas-MacBook-Air OSLab %

```

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }
    if (pid == 0) {
        char *args[] = {"echo", "Hello from child", NULL};
        execvp(args[0], args);
        perror("execvp");
        return 1;
    }
    waitpid(pid, NULL, 0);
    printf("Parent: child completed.\n");
    return 0;
}
```

```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano child.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc child.c -o child
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./child

Hello from child

Parent: child completed.

eeshayuktha@Eeshas-MacBook-Air OSLab %
```

7. Process Synchronization and Child Process Monitoring Using waitpid().

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        sleep(2);
        printf("Child finished.\n");
        return 0;
    }
```

```

}
printf("Parent waiting using waitpid()...\n");
waitpid(pid, NULL, 0);
printf("Child has terminated.\n");
return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano waitpid.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc waitpid.c -o waitpid
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./waitpid

Parent waiting using waitpid()...

Child finished.

Child has terminated.

eeshayuktha@Eeshas-MacBook-Air OSLab % █

```

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution.

CODE:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main() {
    char *path = getenv("PATH");
    char copy[4096];
    if (path == NULL)
        return 1;

    strncpy(copy, path, sizeof(copy) - 1);
    copy[sizeof(copy) - 1] = '\0';

    char *dir = strtok(copy, ":");
    while (dir != NULL) {
        char full[500];
        snprintf(full, sizeof(full), "%s/%s", dir, "ls");
        if (access(full, X_OK) == 0) {
            printf("Executable found: %s\n", full);
            return 0;
        }
        dir = strtok(NULL, ":");
    }
    printf("Command not found.\n");
    return 0;
}

```

```
}
```

```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano path.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc path.c -o path
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./path

Executable found: /bin/ls

eeshayuktha@Eeshas-MacBook-Air OSLab %
```

8. To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    char name[50];
    char *value;

    printf("Enter variable name: ");
    if (scanf("%49s", name) == 1) {
        value = getenv(name);
        if (value != NULL)
            printf("%s=%s\n", name, value);
        else
            printf("Variable is undefined.\n");
    }
    return 0;
}
```

```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano expand.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc expand.c -o expand
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./expand

Enter variable name: USER

USER=eeshayuktha

eeshayuktha@Eeshas-MacBook-Air OSLab %
```

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

CODE:

```
#include <stdio.h>
#include <string.h>
```

```

void help() {
    printf("Built-ins: cd, help, exit\n");
}

int main() {
    char command[50];
    printf("Enter built-in: ");
    if (scanf("%49s", command) == 1) {
        if (strcmp(command, "help") == 0)
            help();
        else if (strcmp(command, "exit") == 0)
            printf("Exit requested.\n");
        else if (strcmp(command, "cd") == 0)
            printf("cd is a built-in command.\n");
        else
            printf("Invalid built-in command.\n");
    }
    return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano builtin.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc builtin.c -o builtin
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./builtin

Enter built-in: help

Built-ins: cd, help, exit

eeshayuktha@Eeshas-MacBook-Air OSLab % █

```

9. To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.

CODE:

```

#include <stdio.h>
#include <unistd.h>
#include <limits.h>

int main() {
    char path[PATH_MAX];
    printf("Enter directory: ");
    if (scanf("%s", path) == 1) {
        if (chdir(path) == -1) {
            perror("chdir");
            return 1;
        }
        if (getcwd(path, sizeof(path)) != NULL)

```

```

    printf("Current directory: %s\n", path);
}
return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano cd.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc cd.c -o cd
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./cd

Enter directory: /

Current directory: /

eeshayuktha@Eeshas-MacBook-Air OSLab %

```

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

CODE:

```

#include <stdio.h>
#include <string.h>

void execute_builtin(const char *cmd) {
    if (strcmp(cmd, "pwd") == 0)
        printf("pwd: current directory command\n");
    else if (strcmp(cmd, "cd") == 0)
        printf("cd: change directory command\n");
    else if (strcmp(cmd, "exit") == 0)
        printf("exit: terminate shell\n");
    else
        printf("Unknown built-in: %s\n", cmd);
}

```

```

int main() {
    char cmd[50];
    printf("Enter built-in: ");
    if (scanf("%49s", cmd) == 1) {
        execute_builtin(cmd);
    }
    return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano builtin2.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc builtin2.c -o builtin2
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./builtin2

Enter built-in: pwd

pwd: current directory command

eeshayuktha@Eeshas-MacBook-Air OSLab %

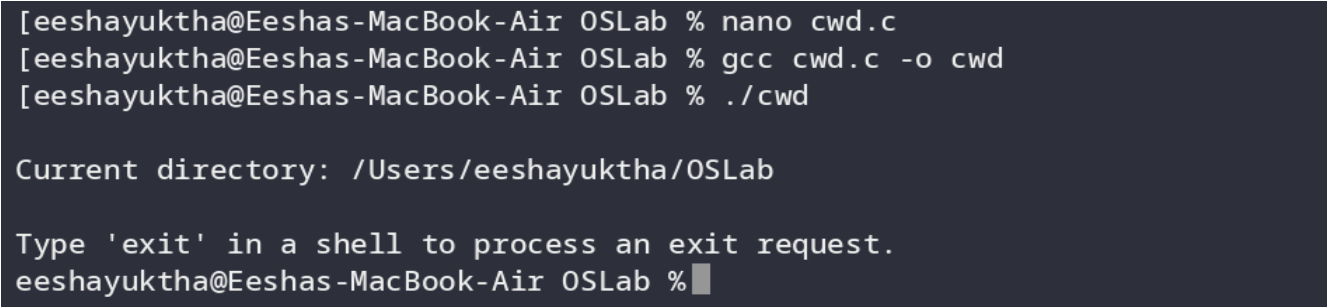
```

10. To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <limits.h>

int main() {
    char cwd[PATH_MAX];
    if (getcwd(cwd, sizeof(cwd)) == NULL) {
        perror("getcwd");
        return 1;
    }
    printf("Current directory: %s\n", cwd);
    printf("Type 'exit' in a shell to process an exit request.\n");
    return 0;
}
```



```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano cwd.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc cwd.c -o cwd
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./cwd

Current directory: /Users/eeshayuktha/OSLab

Type 'exit' in a shell to process an exit request.
eeshayuktha@Eeshas-MacBook-Air OSLab %
```

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    char name[50], value[100];
    printf("Variable name: ");
    if (scanf("%49s", name) != 1) return 1;
    printf("Variable value: ");
    if (scanf("%99s", value) != 1) return 1;

    if (setenv(name, value, 1) == -1) {
        perror("setenv");
        return 1;
    }
}
```

```

printf("Exported: %s=%s\n", name, getenv(name));
return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano export.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc export.c -o export
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./export

Variable name: OSSP

Variable value: Ready

Exported: OSSP=Ready

eeshayuktha@Eeshas-MacBook-Air OSLab % █

```

11. To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

CODE:

```

#include <stdio.h>
#include <string.h>

#define MAX 5

int main() {
    char history[MAX][100];
    int count = 0;
    char cmd[100];
    for (int i = 0; i < MAX; i++) {
        printf("Command %d: ", i + 1);
        fgets(cmd, sizeof(cmd), stdin);
        cmd[strcspn(cmd, "\n")] = '\0';
        strcpy(history[count++], cmd);
    }

    printf("\nHistory:\n");
    for (int i = 0; i < count; i++)
        printf("%d: %s\n", i + 1, history[i]);

    return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano historybuf.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc historybuf.c -o historybuf
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./historybuf
Command 1: ls

Command 2: pwd

Command 3: gcc test.c

Command 4: ./test

Command 5: history

History:

1: ls
2: pwd
3: gcc test.c
4: ./test
5: history

eeshayuktha@Eeshas-MacBook-Air OSLab % █

```

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

CODE:

```

#include <stdio.h>
#include <string.h>

int main() {
    char input[] = "ls -l | grep .c | wc -l";
    char *cmd = strtok(input, "|");
    int n = 1;

    while (cmd != NULL) {
        printf("Command %d: %s\n", n++, cmd);
        cmd = strtok(NULL, "|");
    }

    return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano pipeline.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc pipeline.c -o pipeline
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./pipeline

Command 1: ls -l

Command 2: grep .c

Command 3: wc -l

eeshayuktha@Eeshas-MacBook-Air OSLab % █

```

12. To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

CODE:

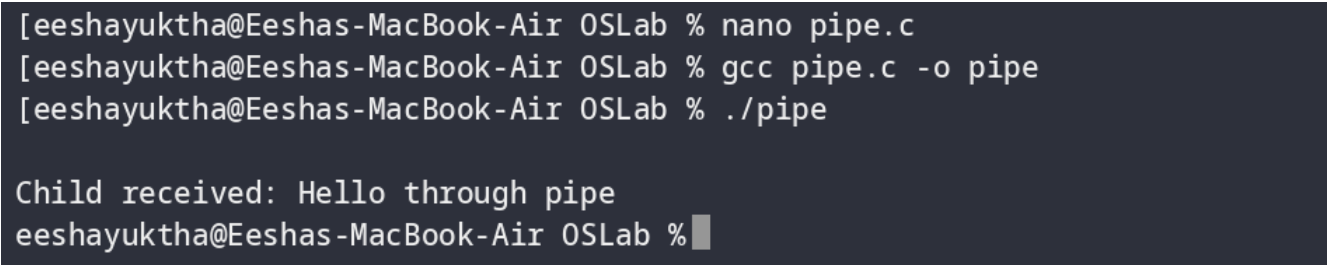
```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int fd[2];
    pipe(fd);

    pid_t pid = fork();

    if (pid == 0) {
        close(fd[1]);
        char buffer[100];
        read(fd[0], buffer, sizeof(buffer));
        printf("Child received: %s\n", buffer);
        close(fd[0]);
    } else {
        close(fd[0]);
        char msg[] = "Hello through pipe";
        write(fd[1], msg, sizeof(msg));
        close(fd[1]);
        wait(NULL);
    }

    return 0;
}
```



```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano pipe.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc pipe.c -o pipe
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./pipe

Child received: Hello through pipe
eeshayuktha@Eeshas-MacBook-Air OSLab %
```

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
```

```

int main() {
    int p1[2], p2[2];
    pipe(p1);
    pipe(p2);

    if (fork() == 0) {
        close(p1[0]);
        close(p2[0]);
        close(p2[1]);
        dprintf(p1[1], "Data from process 1");
        close(p1[1]);
        return 0;
    }

    if (fork() == 0) {
        char buffer[100];
        close(p1[1]);
        close(p2[0]);
        read(p1[0], buffer, sizeof(buffer));
        dprintf(p2[1], "%s -> process 2", buffer);
        close(p1[0]);
        close(p2[1]);
        return 0;
    }

    close(p1[0]); close(p1[1]); close(p2[1]);
    char buffer[100];
    read(p2[0], buffer, sizeof(buffer));
    printf("Final output: %s\n", buffer);
    close(p2[0]);

    wait(NULL);
    wait(NULL);
    return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano multipipe.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc multipipe.c -o multipipe
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./multipipe

```

```

Final output: Data from process 1 -> process 2
eeshayuktha@Eeshas-MacBook-Air OSLab % █

```

13. To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

CODE:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

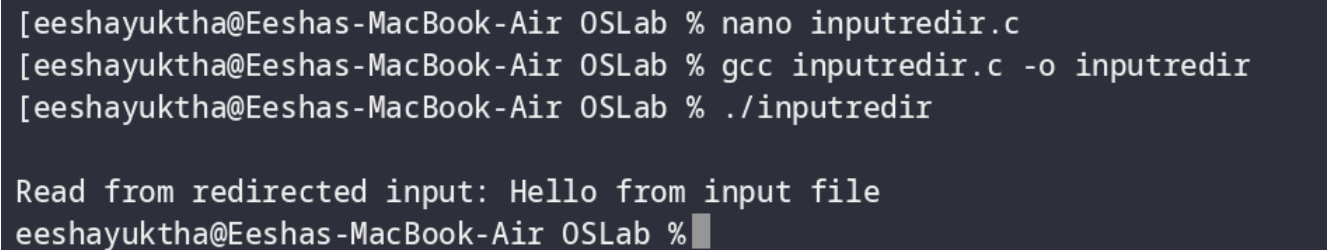
int main() {
    int fd = open("input.txt", O_RDONLY);

    if (fd < 0) {
        perror("input.txt");
        return 1;
    }

    dup2(fd, STDIN_FILENO);
    close(fd);

    char buffer[100];
    if (fgets(buffer, sizeof(buffer), stdin))
        printf("Read from redirected input: %s", buffer);

    return 0;
}
```



```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano inputredir.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc inputredir.c -o inputredir
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./inputredir

Read from redirected input: Hello from input file
eeshayuktha@Eeshas-MacBook-Air OSLab %
```

To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection.

CODE:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd < 0) {
        perror("output.txt");
        return 1;
    }
}
```

```

}

dup2(fd, STDOUT_FILENO);
close(fd);

printf("This text is redirected to output.txt\n");
return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano outputredir.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc outputredir.c -o outputredir
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./outputredir
[eeshayuktha@Eeshas-MacBook-Air OSLab % cat output.txt

output.txt contents:

This text is redirected to output.txt

eeshayuktha@Eeshas-MacBook-Air OSLab %

```

14. To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

CODE:

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

```

```

int main() {
    int fd = open("output.txt", O_WRONLY | O_CREAT | O_APPEND, 0644);

    if (fd < 0) {
        perror("output.txt");
        return 1;
    }

    dup2(fd, STDOUT_FILENO);
    close(fd);

    printf("This line is appended.\n");
    return 0;
}

```

```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano append.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc append.c -o append
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./append
[eeshayuktha@Eeshas-MacBook-Air OSLab % cat output.txt
output.txt contents:

This text is redirected to output.txt

This line is appended.

eeshayuktha@Eeshas-MacBook-Air OSLab %
```

To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases.

CODE:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
```

```
int main() {
    int fd = open("errors.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd < 0)
        return 1;

    dup2(fd, STDERR_FILENO);
    close(fd);

    fprintf(stderr, "This is an error message.\n");
    return 0;
}
```

```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano stderr.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc stderr.c -o stderr
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./stderr
[eeshayuktha@Eeshas-MacBook-Air OSLab % cat errors.txt
errors.txt contents:

This is an error message.

eeshayuktha@Eeshas-MacBook-Air OSLab %
```

15. To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

CODE:

```
#include <stdio.h>
#include <fcntl.h>
```

```

#include <unistd.h>

int main() {
    int fd = open("combined.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd < 0)
        return 1;

    dup2(fd, STDOUT_FILENO);
    dup2(fd, STDERR_FILENO);
    close(fd);

    printf("Standard output message.\n");
    fprintf(stderr, "Standard error message.\n");

    return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano combined.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc combined.c -o combined
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./combined
[eeshayuktha@Eeshas-MacBook-Air OSLab % cat combined.txt
combined.txt contents:

Standard output message.

Standard error message.

eeshayuktha@Eeshas-MacBook-Air OSLab %

```

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.

CODE:

```

#include <stdio.h>
#include <string.h>

int main() {
    char command[] = "cat < input.txt | grep hello > output.txt";

    printf("Command: %s\n", command);
    printf("Input redirection detected.\n");
    printf("Pipeline detected.\n");
    printf("Output redirection detected.\n");
    printf("Execution order: input -> cat -> grep -> output.\n");

    return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano complex.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc complex.c -o complex
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./complex

Command: cat < input.txt | grep hello > output.txt

Input redirection detected.

Pipeline detected.

Output redirection detected.

Execution order: input -> cat -> grep -> output.
eeshayuktha@Eeshas-MacBook-Air OSLab % █

```

16. To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

CODE:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

```

```

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        sleep(5);
        printf("Background job completed.\n");
        return 0;
    }

    printf("Background job started. PID=%d\n", pid);
    printf("Parent returns to prompt immediately.\n");

    waitpid(pid, NULL, 0);
    return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano background.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc background.c -o background
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./background

Background job started. PID=50920

Parent returns to prompt immediately.

Background job completed.

eeshayuktha@Eeshas-MacBook-Air OSLab % █

```

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

struct Job {
    int id;
    pid_t pid;
    char state[20];
};

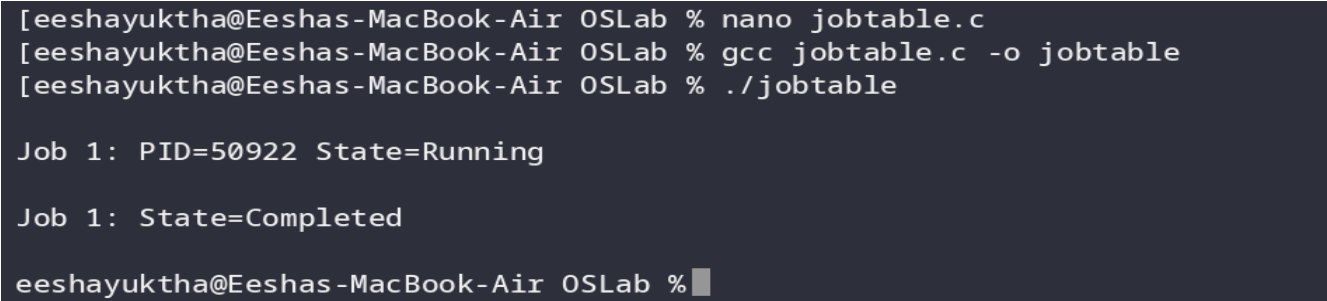
int main() {
    struct Job job;
    pid_t pid = fork();

    if (pid == 0) {
        sleep(2);
        return 0;
    }

    job.id = 1;
    job.pid = pid;
    snprintf(job.state, sizeof(job.state), "Running");
    printf("Job %d: PID=%d State=%s\n", job.id, job.pid, job.state);

    waitpid(pid, NULL, 0);
    snprintf(job.state, sizeof(job.state), "Completed");

    printf("Job %d: State=%s\n", job.id, job.state);
    return 0;
}
```



```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano jobtable.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc jobtable.c -o jobtable
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./jobtable

Job 1: PID=50922 State=Running

Job 1: State=Completed

eeshayuktha@Eeshas-MacBook-Air OSLab %
```

17. To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

CODE:

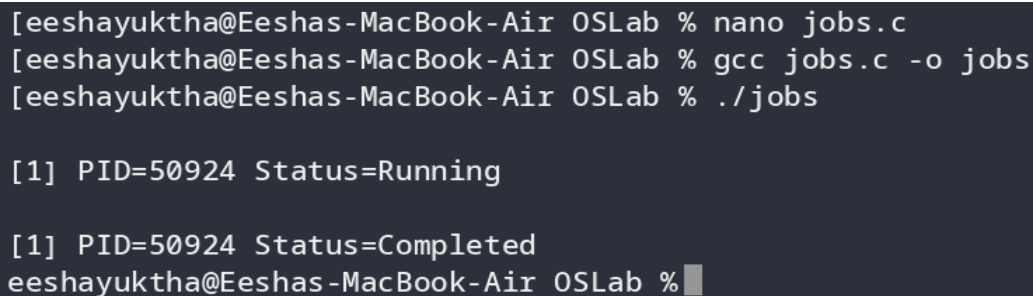
```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        sleep(3);
        return 0;
    }

    printf("[1] PID=%d Status=Running\n", pid);
    waitpid(pid, NULL, 0);
    printf("[1] PID=%d Status=Completed\n", pid);

    return 0;
}
```

A terminal window with a dark background. The prompt is [eeshayuktha@Eeshas-MacBook-Air OSLab %]. The user enters 'nano jobs.c', then 'gcc jobs.c -o jobs', and finally './jobs'. The output shows '[1] PID=50924 Status=Running' followed by '[1] PID=50924 Status=Completed'. The prompt returns to [eeshayuktha@Eeshas-MacBook-Air OSLab %].

```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano jobs.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc jobs.c -o jobs
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./jobs

[1] PID=50924 Status=Running

[1] PID=50924 Status=Completed
eeshayuktha@Eeshas-MacBook-Air OSLab %
```

To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching.

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Job running in foreground. PID=%d\n", getpid());
        sleep(2);
        return 0;
    }

    printf("Giving foreground control to job PID=%d\n", pid);
```

```

waitpid(pid, NULL, 0);
printf("Foreground job completed.\n");

return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano fg.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc fg.c -o fg
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./fg

Giving foreground control to job PID=50926

Job running in foreground. PID=50926

Foreground job completed.
eeshayuktha@Eeshas-MacBook-Air OSLab %

```

18. To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

CODE:

```

#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig) {
    printf("Received signal: %d\n", sig);
}

int main() {
    signal(SIGUSR1, handler);

    printf("PID: %d\n", getpid());
    printf("Send SIGUSR1 using: kill -USR1 PID\n");

    while (1)
        pause();

    return 0;
}

```

```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano signal.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc signal.c -o signal
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./signal
[eeshayuktha@Eeshas-MacBook-Air OSLab % kill -USR1 50928

PID: 50928

Send SIGUSR1 using: kill -USR1 PID

Received signal: 30

eeshayuktha@Eeshas-MacBook-Air OSLab %
```

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery.

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child running. PID=%d\n", getpid());
        raise(SIGSTOP);
        printf("Child resumed.\n");
        return 0;
    }

    sleep(1);
    printf("Sending SIGCONT to child.\n");
    kill(pid, SIGCONT);

    waitpid(pid, NULL, 0);
    printf("Job completed.\n");

    return 0;
}
```

```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano resume.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc resume.c -o resume
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./resume

Child running. PID=50930

Sending SIGCONT to child.

Child resumed.

Job completed.

eeshayuktha@Eeshas-MacBook-Air OSLab %
```

19. To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

CODE:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
```

```
void handler(int sig) {
    printf("\nSignal %d received. Program is stable.\n", sig);
}
```

```
int main() {
    struct sigaction sa;

    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTERM, &sa, NULL);

    printf("PID: %d\n", getpid());

    while (1)
        pause();

    return 0;
}
```

```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano sigaction.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc sigaction.c -o sigaction
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./sigaction

^C

PID: 50932

^C

Signal 2 received. Program is stable.

eeshayuktha@Eeshas-MacBook-Air OSLab %
```

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

CODE:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>
```

```
pid_t child;
```

```
void handler(int sig) {
    if (child > 0)
        kill(child, SIGINT);
}
```

```
int main() {
    child = fork();

    if (child == 0) {
        while (1) {
            printf("Foreground job running...\n");
            sleep(1);
        }
    }
}
```

```
signal(SIGINT, handler);
waitpid(child, NULL, 0);
```

```
printf("Foreground job terminated.\n");
return 0;
}
```

```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano forward.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc forward.c -o forward
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./forward

^C

Foreground job running...

Foreground job running...

^C

Foreground job terminated.

eeshayuktha@Eeshas-MacBook-Air OSLab %
```

20. To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

CODE:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
```

```
void handler(int sig) {
    printf("\nSIGTSTP received. Process will stop.\n");
    signal(SIGTSTP, SIG_DFL);
    raise(SIGTSTP);
}
```

```
int main() {
    signal(SIGTSTP, handler);

    printf("PID: %d\n", getpid());
    printf("Press Ctrl+Z to suspend.\n");

    while (1) {
        printf("Running...\n");
        sleep(1);
    }

    return 0;
}
```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano sigtstp.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc sigtstp.c -o sigtstp
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./sigtstp

^Z

PID: 50936

Press Ctrl+Z to suspend.

Running...

Running...

^Z

SIGTSTP received. Process will stop.

[1]+  Stopped ./sigtstp
eeshayuktha@Eeshas-MacBook-Air OSLab %

```

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

CODE:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

```

```

int main() {
    printf("PID : %d\n", getpid());
    printf("PGID : %d\n", getpgrp());

    if (setpgid(0, 0) == -1)
        perror("setpgid");
    else
        printf("New PGID: %d\n", getpgrp());

    return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano pgrp.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc pgrp.c -o pgrp
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./pgrp

PID : 50938

PGID : 50900

New PGID: 50938

eeshayuktha@Eeshas-MacBook-Air OSLab %

```

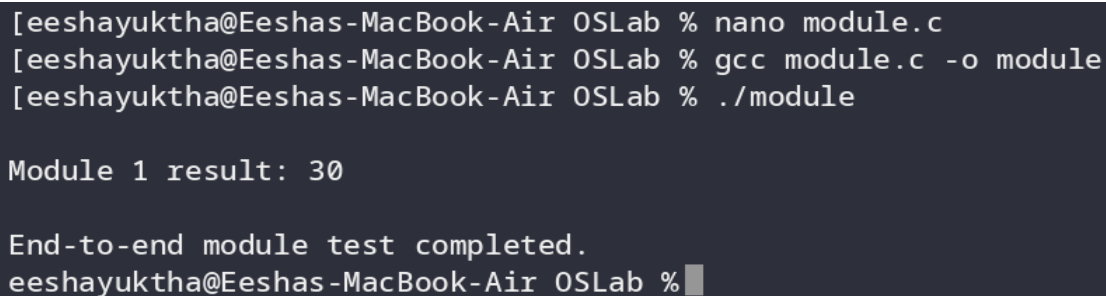
21. To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

CODE:

```
#include <stdio.h>
```

```
int add(int a, int b) {  
    return a + b;  
}
```

```
int main() {  
    int a = 10, b = 20;  
  
    printf("Module 1 result: %d\n", add(a, b));  
    printf("End-to-end module test completed.\n");  
  
    return 0;  
}
```



```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano module.c  
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc module.c -o module  
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./module  
  
Module 1 result: 30  
  
End-to-end module test completed.  
eeshayuktha@Eeshas-MacBook-Air OSLab %
```

To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {  
    FILE *fp = fopen("missing.txt", "r");  
  
    if (fp == NULL) {  
        perror("Runtime error");  
        printf("Recovering gracefully...\n");  
        return 0;  
    }  
  
    fclose(fp);  
    return 0;  
}
```

```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano runtime.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc runtime.c -o runtime
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./runtime

Runtime error: No such file or directory

Recovering gracefully...

eeshayuktha@Eeshas-MacBook-Air OSLab %
```

22. To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
    int *ptr = malloc(10 * sizeof(int));

    if (ptr == NULL)
        return 1;

    for (int i = 0; i < 10; i++)
        ptr[i] = i;

    printf("Memory allocated successfully.\n");

    free(ptr);
    printf("Memory released successfully.\n");

    return 0;
}
```

```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano memory.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc memory.c -o memory
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./memory

Memory allocated successfully.

Memory released successfully.

eeshayuktha@Eeshas-MacBook-Air OSLab %
```

To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

CODE:

```
#include <stdio.h>
```

```

int main() {
    int tests[] = {1, 2, 3};
    int expected[] = {2, 4, 6};

    for (int i = 0; i < 3; i++) {
        int result = tests[i] * 2;

        printf("Test %d: Input=%d Expected=%d Result=%d ",
            i + 1, tests[i], expected[i], result);

        if (result == expected[i])
            printf("PASS\n");
        else
            printf("FAIL\n");
    }

    return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano tests.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc tests.c -o tests
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./tests

Test 1: Input=1 Expected=2 Result=2 PASS

Test 2: Input=2 Expected=4 Result=4 PASS

Test 3: Input=3 Expected=6 Result=6 PASS
eeshayuktha@Eeshas-MacBook-Air OSLab % █

```

23. To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance.

CODE:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

int main() {
    clock_t start = clock();

    for (int i = 0; i < 3; i++) {
        if (fork() == 0) {
            printf("Pipeline job %d, PID=%d\n", i + 1, getpid());
            sleep(1);
            return 0;
        }
    }
}

```

```

    }
}

for (int i = 0; i < 3; i++)
    wait(NULL);

clock_t end = clock();

printf("All jobs completed.\n");
printf("CPU time: %.3f seconds\n",
    (double)(end - start) / CLOCKS_PER_SEC);

return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano largepipe.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc largepipe.c -o largepipe
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./largepipe

Pipeline job 1, PID=50940

Pipeline job 2, PID=50941

Pipeline job 3, PID=50942

All jobs completed.

CPU time: 0.001 seconds

eeshayuktha@Eeshas-MacBook-Air OSLab % █

```

To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation.

CODE:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    printf("Shell Architecture:\n");
    printf("Input -> Parser -> Process Creation -> exec() -> wait()\n");
    printf("Example syscalls: fork(), execvp(), waitpid(), pipe(), dup2()\n");

    pid_t pid = fork();

    if (pid == 0) {
        printf("Execution path: Child process\n");
        return 0;
    }
    wait(NULL);
}

```

```

printf("Execution path: Parent process\n");

return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano architecture.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc architecture.c -o architecture
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./architecture

Shell Architecture:

Input -> Parser -> Process Creation -> exec() -> wait()

Example syscalls: fork(), execvp(), waitpid(), pipe(), dup2()

Execution path: Child process

Execution path: Parent process

eeshayuktha@Eeshas-MacBook-Air OSLab %

```

24. To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables.

CODE:

```
#include <stdio.h>
```

```

int main() {
    printf("Project structure check\n");
    printf("src/ - source files\n");
    printf("include/ - header files\n");
    printf("build/ - compiled files\n");
    printf("README.md - project documentation\n");
    printf("Makefile - build instructions\n");
    return 0;
}

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano structure.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc structure.c -o structure
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./structure

Project structure check

src/ - source files

include/ - header files

build/ - compiled files

README.md - project documentation

Makefile - build instructions

eeshayuktha@Eeshas-MacBook-Air OSLab %

```

To Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review.

CODE:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig) {
    printf("Demo signal received: %d\n", sig);
}

int main() {
    signal(SIGUSR1, handler);

    printf("OSSP Demo\n");
    printf("PID: %d\n", getpid());
    printf("Feature 1: Process execution\n");
    printf("Feature 2: Pipelines\n");
    printf("Feature 3: Signal handling\n");

    printf("Sending demonstration signal...\n");
    raise(SIGUSR1);

    printf("Final project review completed.\n");
    return 0;
}
```

```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano demo.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc demo.c -o demo
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./demo
OSSP Demo

PID: 50944

Feature 1: Process execution

Feature 2: Pipelines

Feature 3: Signal handling

Sending demonstration signal...

Demo signal received: 30

Final project review completed.
eeshayuktha@Eeshas-MacBook-Air OSLab % █
```